

33

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1407	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	V. Santhosh Kumar	Reg. No.:	192411015
Section / Batch:		Date:	16/7/26

Code of Conduct

(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Santhosh

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	3 / 4	4 / 4	9 / 4	3 / 4	91.25
2	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
3	3 / 4	4 / 4	3 / 4	4 / 4	4 / 4	88.75
4	4 / 4	4 / 4	3 / 4	3 / 4	3 / 4	85
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	89
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. D. H

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a) b) c) d) e)
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b="abc"; in t c = 30; ch ar d ="xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of : i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0's and the number of 1's are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1's.
Consider a lexical analyzer for a simplified programming language. The language consists	

5

of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, :, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

`int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }`

Questions:

- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

COMPILER DESIGN

ANALYTICAL PROBLEM SOLVING

V. SANTHOSH
KUNAR
19/11/2015
CSE

1. Trace the processing of the input expression
 $P = (a * b) + (b * c) * 2$ through all phases of compiler

Trace the processing

$$P = (a * b) + (b * c) * 2$$

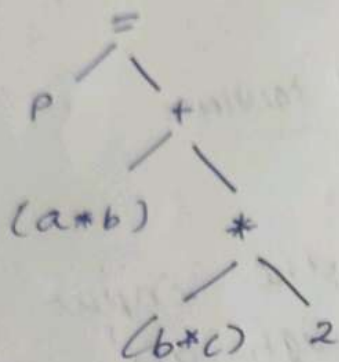
(a) Tokens generated

Lexeme	Tokens
P	identifier (id)
=	assignment operator
(	Left Parenthesis
a	identifier
*	Multiplication operator
b	identifier
*	Multiplication
2	Number

Tokens :

$\langle P \rangle \Rightarrow \langle (\rangle \langle a \rangle \langle * \rangle \langle b \rangle \langle + \rangle \langle (\rangle \langle b \rangle \langle * \rangle \langle c \rangle$
 $\langle id_1 \rangle \Rightarrow \langle (\rangle \langle id_1 \rangle \langle * \rangle \langle id_3 \rangle \langle + \rangle \langle (\rangle \langle id_3 \rangle \langle$

Syntax tree



(C) Intermediate code

$t_1 = a * b$
 $t_2 = b * c$
 $t_3 = t_2 * 2$
 $t_4 = t_1 + t_3$
 $P = t_4$

Target code (e)

LOAD R1, a
 MUL R1, b
 LOAD R2, b
 MUL R2, c
 MUL R2, 2
 ADD R1, R2
 STORE P, R1

(2.)

a) $(\epsilon + a a^*)^*$
accepted string

- ϵ
- a
- aa
- aaa
- aaaa

b) $(a^* + b^*)^* a b b$

Accepted string

- a b b
- a a b b
- b a b b
- a a a b b

c) $(a^* b^*)^* a b a (a^* + b^*)^*$

Accepted string

- a b a
- a a b a
- a b a a
- a b a b a
- b a a b a

d) $(a + a b)^*$

Accepted string

- ϵ
- a
- a b
- a a
- a a b

Find the number of tokens in C program.

S.NO	Token
1	main
2	C
3	)
4	{
5	int
6	a
7	=
8	10
9	'
10	char
11	b
12	=
13	abc
14	'
15	int
16	C
17	=

18	30
19	}
20	char
21	d
22	=
23	"xyz"
25	int
26	n
27	=
28	40.5

(4) i) Language of all strings having an even number of 0's

$$\text{Recurrence equation} = 1 * (01^* 01^*)^*$$

(ii) Language of all strings containing at least one 1

$$R.E = (0+1)^* 1 (0+1)^*$$

(iii) Language of all strings in which both the number of 0's and number of 1's are odd

$$R.E = (00+11)^* (01+10) (00+11)^* (01+10) (00+11)^*$$

(iv) Language of all strings with 1 and write 0

$$R \cdot E = 1(0+1)^* 0$$

(v) Language of all strings that do not contain consecutive 1s

$$R \cdot E = (0+10)^* (E+1)$$

⑤

John n. 40	John
1	int
2	var 1 2 3
3	C
4	=
5	1 2 3 4 5
6	;
7	float
8	var 4 5 6
9	=
10	1 2 3 4 5
11	;
12	if

13	C
14	var 1 2 3
15	= =
16	var 4 5 6
17	}
18	{
19	return
20	}

b.) Categorized token

1) categorized token

keyword :

int return
float
if
